

 **ATTENTION — DOCUMENT EN CONSTRUCTION**

BROUILLON — TRAVAIL EN COURS

Ce document est actuellement en cours de développement.

Il peut contenir des informations incomplètes, provisoires ou susceptibles d'être modifiées.

NE PAS CONSIDÉRER CE DOCUMENT COMME LA SPÉCIFICATION TECHNIQUE FINALE DU DECScale SOFT IP.

 **WARNING — WORK IN PROGRESS**

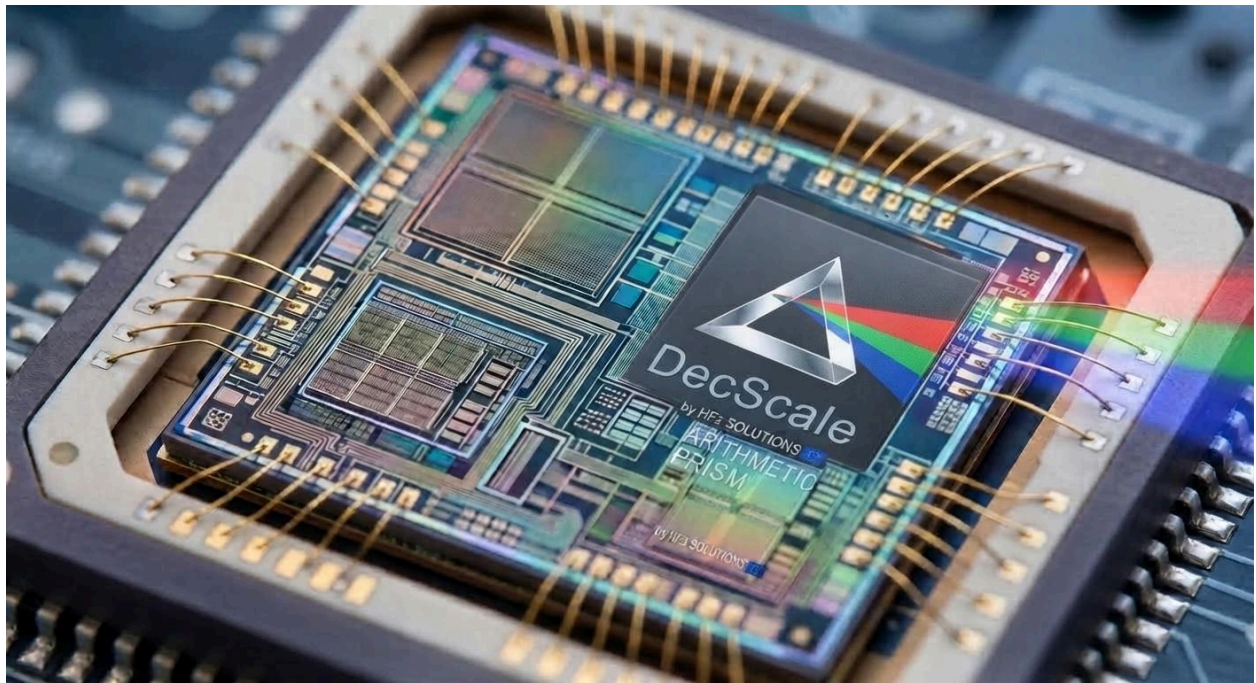
DRAFT DOCUMENT — UNDER CONSTRUCTION

This document is currently under development.

It may contain incomplete, provisional, or subject-to-change information.

DO NOT CONSIDER THIS DOCUMENT AS THE FINAL TECHNICAL SPECIFICATION OF THE DecScale SOFT IP.

hamdi.fouzai@gmail.com



DecScale Soft IP Description

05.09.2026

Hamdi Fouzai

HFz Solutions
Arithmetic Prisms
13 Rue Christophe Thomas Walliser
67200 Strasbourg

Overview

The DecScale is a hardware arithmetic-assistance circuit designed to support a 32-bit System Master (hereafter referred to simply as the Master). Its primary role is to handle multiplications and divisions by constant factors, enabling the system to directly handle decimal scales.

The task of the DecScale is to provide the Master with a set of products and quotients, which can be used as mantissas to derive decimal scaling coefficients.

These mantissas correspond to the products $\times 5$, $\times 25$, $\times 125$ and the quotients $/5$, $/25$, $/125$. In addition, the decimal scaling factors $\times 10$, $\times 100$, $\times 1000$ and $/10$, $/100$, $/1000$ are directly derived and made immediately available to the Master.

This enables the Master to derive other composite scaling factors and quotients—such as $\times 50$, $\times 250$, $\times 500$, as well as fractional scales like $\times 0.2$, $\times 0.004$, $\times 0.002$, etc.—using simple binary shifts.

Additionally, the circuit provides auxiliary operations including $/3$, $\times -1$, and the remainders (modulos) associated with the division outputs.

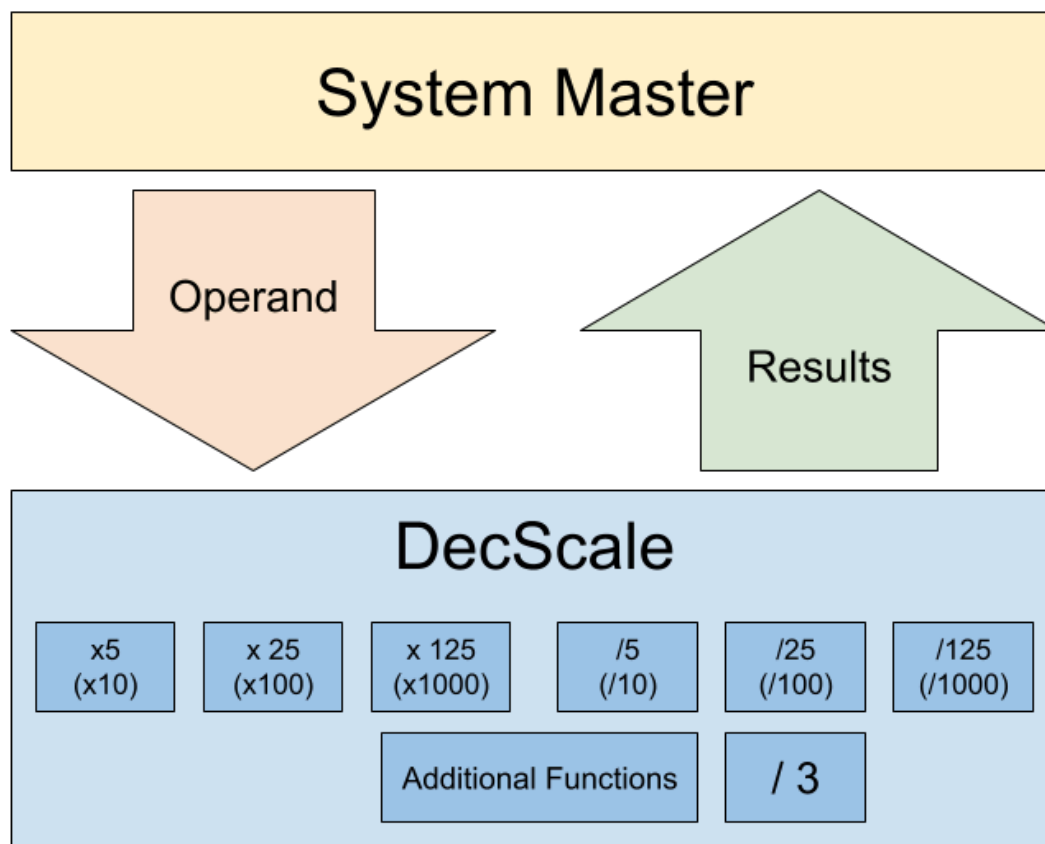


Figure 1: DecScale High-Level Principle Diagram

Primitive Derivation by Binary Scaling

The following tables illustrate the scaling coefficients derived from the DecScale primitive operations through binary scaling.

1. Primitive Derivation by Left Shifting

Each left-shift level doubles the corresponding scaling coefficient.

	<< 1	<< 2	<< 3	<< 4	<< 5	<< 6	<< 7	<< 8
x5	x10	x20	x40	x80	x160	x320	x640	x1 280
x10	x20	x40	x80	x160	x320	x640	x1 280	x2 560
x25	x50	x100	x200	x400	x800	x1 600	x3 200	x6 400
x100	x200	x400	x800	x1 600	x3 200	x6 400	x12 800	x25 600
x125	x250	x500	x1 000	x2 000	x4 000	x8 000	x16 000	x32 000
x1000	x2000	x4 000	x8 000	x16 000	x32 000	x64 000	x128 000	x256 000
/5	x0.4	x0.8	x1.6	x3.2	x6.4	x12.8	x25.6	x51.2
/10	x0.2	x0.4	x0.8	x1.6	x3.2	x6.4	x12.8	x25.6
/25	x0.08	x0.16	x0.32	x0.64	x1.28	x2.56	x5.12	x10.24
/100	x0.02	x0.04	x0.08	x0.16	x0.32	x0.64	x1.28	x2.56
/125	x0.016	x0.032	x0.064	x0.128	x0.256	x0.512	x1.024	x2.048
/1000	x0.002	x0.004	x0.008	x0.016	x0.032	x0.064	x0.128	x0.256
x -1	x(-2)	x(-4)	x(-8)	x(-16)	x(-32)	x(-64)	x(-128)	x(-256)
/3 *	x2/3 x0.666	x4/3 x1.333	x8/3 x2.666	x16/3 x5.333	x32/3 x10.666	x64/3 x21.333	x128/3 x42.666	x256/3 x85.333

*The /3 family is shown in decimal form for consistency; its exact coefficients are $(2^n/3)$.

The highlighted coefficients are of particular interest for scaling applications based on an extended decimal representation.

2. Primitive Derivation by Right Shifting

Each right-shift level halves the corresponding scaling coefficient.

	>> 1	>> 2	>> 3	>> 4	>> 5	>> 6	>> 7	>> 8
x5	x5/2 x2.5	x5/4 x1.25	x5/8 x0.625	x5/16 x0.3125	x5/32 x0.15625	x5/64 x0.078125	x5/128 x0.0390625	x5/256 x0.01953125
x10	x10/2 x5	x10/4 x2.5	x10/8 x1.25	x10/16 x0.625	x10/32 x0.3125	x10/64 x0.15625	x10/128 x0.078125	x10/256 x0.0390625
x25	x25/2 x12.5	x25/4 x6.25	x25/8 x3.125	x25/16 x1.5625	x25/32 x0.78125	x25/64 x0.390625	x25/128 x0.1953125	x25/256 x0.09765625
x100	x100/2 x50	x100/4 x25	x100/8 x12.5	x100/16 x6.25	x100/32 x3.125	x100/64 x1.5625	x100/128 x0.78125	x100/256 x0.390625
x125	x125/2 x62.5	x125/4 x31.25	x125/8 x15.625	x125/16 x7.8125	x125/32 x3.90625	x125/64 x1.953125	x125/128 x0.9765625	x125/256 x0.48828125
x1000	x1000/2 x500	x1000/4 x250	x1000/8 x125	x1000/16 x62.5	x1000/32 x31.25	x1000/64 x15.625	x1000/128 x7.8125	x1000/256 x3.90625
/5 x0.2	/10 x0.1	/20 x0.05	/40 x0.025	/80 x0.0125	/160 x0.00625	/320 x0.003125	/640 x0.0015625	/1280 x7.8125×10 ⁻⁴
/10 x0.1	/20 x0.05	/40 x0.025	/80 x0.0125	/160 x0.00625	/320 x0.003125	/640 x0.0015625	/1280 x7.8125×10 ⁻⁴	/2560 x3.90625×10 ⁻⁴
/25 x0.04	/50 x0.02	/100 x0.01	/200 x0.005	/400 x0.0025	/800 x0.00125	/1600 x6.25×10 ⁻⁴	/3200 x3.125×10 ⁻⁴	/6400 x1.5625×10 ⁻⁴
/100 x0.01	/200 x0.005	/400 x0.0025	/800 x0.00125	/1600 x6.25×10 ⁻⁴	/3200 x3.125×10 ⁻⁴	/6400 x1.5625×10 ⁻⁴	/12800 x7.8125×10 ⁻⁵	/25600 x3.90625×10 ⁻⁵
/125 x0.008	/250 x0.004	/500 x0.002	/1000 x0.001	/2000 x0.0005	/4000 x0.00025	/8000 x1.25×10 ⁻⁴	/16000 x6.25×10 ⁻⁵	/32000 x3.125×10 ⁻⁵
/1000 x0.001	/2000 x0.0005	/4000 x0.00025	/8000 x1.25×10 ⁻⁴	/16000 x6.25×10 ⁻⁵	/32000 x3.125×10 ⁻⁵	/64000 x1.5625×10 ⁻⁵	/128000 x7.8125×10 ⁻⁶	/256000 x3.90625×10 ⁻⁶
x(-1)	x(-1/2) x-0.5	x(-1/4) x-0.25	x(-1/8) x-0.125	x(-1/16) x-0.0625	x(-1/32) x-0.03125	x(-1/64) x-0.015625	x(-1/128) x-0.0078125	x(-1/256) x-0.00390625
/3	/6 x0.166..	/12 x0.083..	/24 x0.0416...	/48 x0.0208...	/96 x0.0104...	/192 x0.0052...	/384 x0.002604...	/768 x0.001302...

Scope, Purpose & Integration

This document is not intended to be a low-level datasheet. Rather, it provides a high-level functional description of the DecScale Soft IP and its operating principles.

Integration into a specific host environment requires an external wrapper to adapt the DecScale signals to the target System Master's bus interface and specifications.

Nevertheless, the Soft IP core remains highly versatile and adaptable across a broad range of architectures. While this document uses the 32-bit implementation as the baseline, future roadmap variants include 32/64-bit and full 64-bit architectures, while maintaining the same foundational principles.

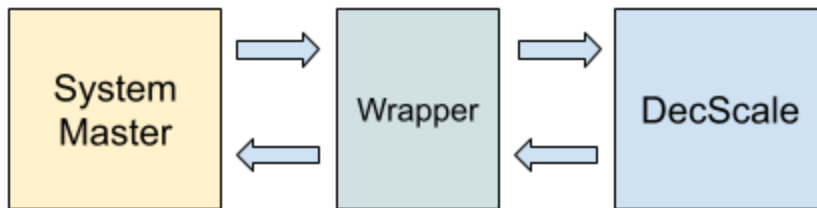


Figure 2: DecScale Integration and Wrapper Interface

Core signals

- **Clock (*i_clk* / Clock):** Synchronizes the wrapper's input capture registers and control state machine to match the Master's bus timing.
- **Command (*i_cmd* / Command 16 bits):** Dedicated control register specifying the target arithmetic operation, configuration modes, and interrupt masking options (e.g., stopping or disabling signaling). By convention, this register must remain stable during active transactions.
- **Operand (*i_operand* / Operand 32 bits):** Input data register. The DecScale selectively captures new operands based on its internal state (e.g., only when ready or cleared), preventing unauthorized re-triggers while a calculation or data retention phase is active.
- **Result (*o_result* / Result 32 bits):** Comprehensive read-only output register containing not only the computed data mantissa but also critical contextual metadata, including execution flags (*busy/done*), user-defined tags, and the *MSB exists* flag.
- **Interrupt / Status (*o_irq* / Interrupt):** Configurable interrupt signaling line used to notify the Master of transaction completion. The interrupt can be muted through the *NO_IRQ* control bit of *i_cmd*.

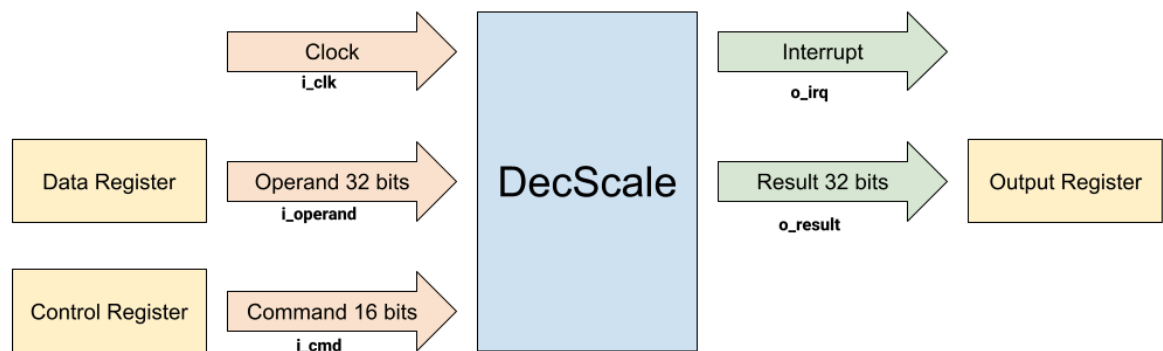


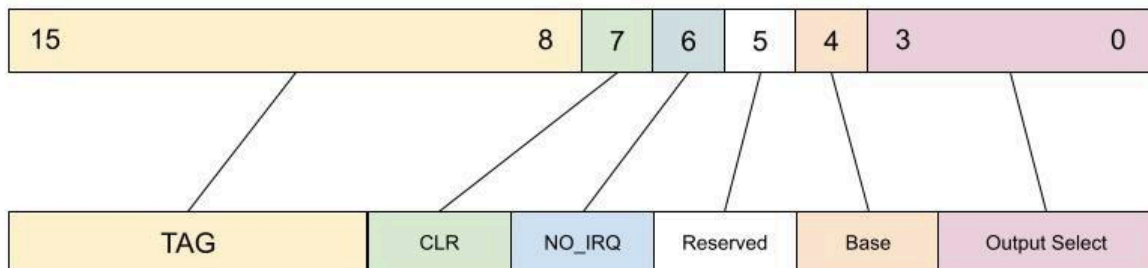
Figure 3: DecScale Soft IP High-Level Block Diagram and MMIO Wrapper Interface, showing functional signal labels paired with their corresponding hardware port definitions (*i_* / *o_* prefixes) and peripheral register boundaries.

Command Word

The DecScale is controlled through the 16-bit **i_cmd** command word, which is structured as follows:

- **[0:3] SELECT** Selects one of the 16 result words available from the DecScale, including arithmetic and auxiliary information. The following diagram provides a more detailed breakdown of these four bits.
- **[4] BASE** Selects between the base-5 and base-10 output sets. **BASE = 0** selects the base-5 (quinary) outputs, while **BASE = 1** selects the base-10 (decimal) outputs.
- **[5] RESERVED** Reserved for future use. This bit has no current function and should be written as **0**
- **[6] NO_IRQ** Mutes the **o_irq** interrupt signal when asserted.
- **[7] CLR** Initializes, unlocks, and re-arms the DecScale for a new result access cycle, according to the operational protocol described later.
- **[8:15] TAG** An optional 8-bit user-defined value provided by the Master. The DecScale can retain and return this value with the associated result.

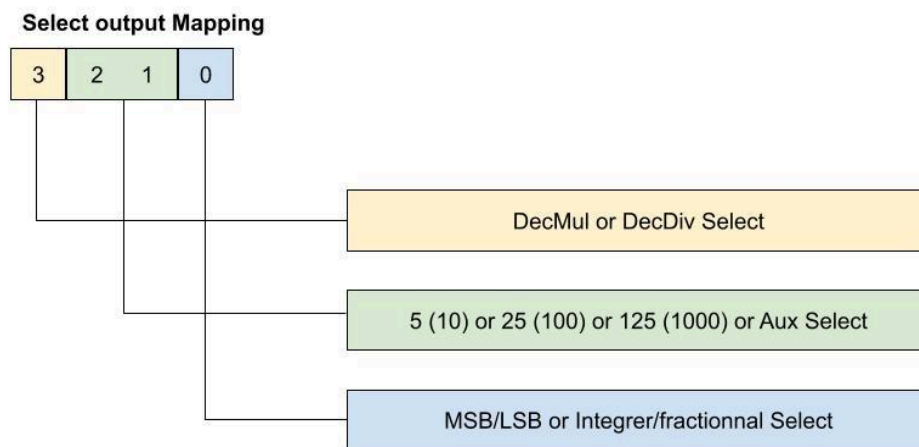
i_cmd : Command Mapping and Tag



Output Select Overview

The 4-bit **Output Select** field allows the Master to navigate through the 16 result words available from the DecScale. These words can be read at any time, regardless of whether the underlying operations have completed.

While an operation is in progress, its availability status is indicated by a **BUSY** flag located in the corresponding result word. Once the operation has completed and its results are valid for reading, the same result word indicates completion through its **DONE** flag.



DecMul or DecDiv Select Bit

The DecScale contains two parallel arithmetic cores:

- **DecMul** handles the multiplication paths $\times 5$, $\times 25$, and $\times 125$, together with their directly derived $\times 10$, $\times 100$, and $\times 1000$ paths. The core has its own **DONE** signal; when asserted, all three multiplication operations are complete and their corresponding results are available. In addition to these arithmetic operations, DecMul also provides auxiliary functions including **TAG retrieval**, **retrieval of the operated input**, and the **two's complement of the input**.
- **DecDiv** handles the division paths and is slightly slower than DecMul because it also generates the corresponding fractional bits. Like DecMul, it has its own **DONE** and **BUSY** signals. The $/3$ operation is an exception: due to the available result-word space, its corresponding integer result does not include a **BUSY** flag

Remark: The `o_irq` interrupt is asserted only when all operations performed by both DecMul and DecDiv have completed. It therefore acts as a global completion signal for the DecScale

Factor Select Bits

The two **Factor Select** bits determine which scaling factor group is accessed within the selected arithmetic core:

- `00` → $\times 5$ / $\times 10$ or $/5$ / $/10$
- `01` → $\times 25$ / $\times 100$ or $/25$ / $/100$
- `10` → $\times 125$ / $\times 1000$ or $/125$ / $/1000$
- `11` → Auxiliary functions

The actual output available for each selection is further determined by the **LSB/MSB** or **Integer/Fractional** select bit

LSB/MSB or Integer/Fractional Select Bit

The **LSB/MSB** or **Integer/Fractional** select bit determines which portion of the selected result information is read by the Master:

- `0` → Selects the **LSB** information for DecMul or the **integer** information for DecDiv. This is where the **DONE** and **BUSY** flags are provided, where applicable.
- `1` → Selects the **MSB** information for DecMul or the **fractional** information for DecDiv.

Output Word Summary

Select		Main Information	Other information	Busy Flag	Done Flag
0000	0	Get LSB of x5 or x10	MSB_EXISTS flag	Yes	Yes
0001	1	Get the remaining MSB of x5 or x10	Retrieve 8-bit TAG	No	No
0010	2	Get LSB of x25 or x100	MSB_EXISTS flag	Yes	Yes
0011	3	Get the remaining MSB of x25 or x100		No	No
0100	4	Get LSB of x125 or x1000	MSB_EXISTS flag	Yes	Yes
0101	5	Get the remaining MSB of x125 or x1000		No	No
0110	6	Retrieve captured operand of the calculations		No	No
0111	7	Get two's complement of captured operand		No	No
1000	8	Get the /5 or /10 integer quotient		Yes	Yes
1001	9	Get the /5 or /10 fractional quotient	Additional fractional precision for /125 or /1000 beyond 32 bits	No	No
1010	10	Get the /25 or /100 integer quotient		Yes	Yes
1011	11	Get the /25 or /100 fractional quotient		No	No
1100	12	Get the /125 or /1000 integer quotient		Yes	Yes
1101	13	Get the /125 or /1000 fractional quotient		No	No
1110	14	Get the /3 integer quotient		No !!	Yes
1111	15	Get the /3 fractional quotient	MOD /3 /5 /25 and /125	No	No

Output Words from DecMul (1/4)

Read Word 0000

Read Word 0000 : Get x5 or x10 LSB Products

31	30	29	28																										0						
Done Flag				Busy Flag				MSB exists Flag				29 Bits LSB Results of x5 or x10																							

BASE = 0 → Get x5 LSB result

BASE = 1 → Get x10 LSB result

Bits [0:28] — LSB: 29-bit LSB portion of the x5 or x10 product, depending on the selected BASE.

Bit [29] — MSB_EXISTS: Indicates that the corresponding product exceeds 29 bits. When asserted, the result must be completed by reading the corresponding MSB word and concatenating both parts to reconstruct the full product.

Bit [30] — BUSY: Indicates that DecMul is still processing the operation. The data currently read must not be considered valid and the Master should read the word again later.

Bit [31] — DONE: Indicates that DecMul has completed its processing, including the corresponding results for the other multiplication factors.

Read Word 0001

Read Word 0001 : Get x5 or x10 MSB (when required) and Retrieve TAG

31	24	23	7	6	5	0
8 Bits retrieve TAG		25 bits padded with zeros			6 Bits MSB Result of x5 or 7 Bits MSB Result of x10	

BASE = 0 → Get x5 MSB result

BASE = 1 → Get x10 MSB result

Bits [6:0] — MSB: Contains the remaining MSB portion of the x5 or x10 product. The number of valid bits depends on the selected BASE: 6 bits for x5 or 7 bits for x10.

Bits [23:7] — Zero padding: 17 bits fixed to 0 and carrying no information.

Bits [31:24] — TAG: Returns the 8-bit TAG associated with the captured calculation.

Output Words from DecMul (2/4)

Read Word 0010

Read Word 0010 : Get x25 or x100 LSB Products

31	30	29	28	0
Done Flag	Busy Flag	MSB exists Flag	29 Bits LSB Results of x25 or x100	

BASE = 0 → Get x25 LSB result

BASE = 1 → Get x100 LSB result

Bits [28:0] — LSB: 29-bit LSB portion of the x25 or x100 product, depending on the selected BASE.

Bit [29] — MSB_EXISTS: Indicates that the corresponding product exceeds 29 bits. When asserted, the result must be completed by reading the corresponding MSB word and concatenating both parts to reconstruct the full product.

Bit [30] — BUSY: Indicates that DecMul is still processing the operation. The data currently read must not be considered valid and the Master should read the word again later.

Bit [31] — DONE: Indicates that DecMul has completed its processing, including the corresponding results for the other multiplication factors.

Read Word 0011

Read Word 0011 : Get x25 or x100 MSB (when required)

31	10	9	7	0
22 bits padded with zeros		8 Bits MSB Result of x25 or 10 Bits MSB Result of x100		

BASE = 0 → Get x25 MSB result

BASE = 1 → Get x100 MSB result

Bits [9:0] — MSB: Contains the remaining MSB portion of the x25 or x100 product. The number of valid bits depends on the selected BASE: 8 bits for x25 or 10 bits for x100.

Bits [31:10] — Zero padding: 22 bits fixed to 0 and carrying no information.

Output Words from DecMul (3/4)

Read Word 0100

Read Word 0100 : Get **x125** or **x1000** LSB Products

31	30	29	28	0
Done Flag	Busy Flag	MSB exists Flag	29 Bits LSB Result of x125 or x1000	

BASE = 0 → Get ×125 LSB result

BASE = 1 → Get ×1000 LSB result

Bits [28:0] — LSB: 29-bit LSB portion of the ×125 or ×1000 product, depending on the selected BASE.

Bit [29] — MSB_EXISTS: Indicates that the corresponding product exceeds 29 bits. When asserted, the result must be completed by reading the corresponding MSB word and concatenating both parts to reconstruct the full product.

Bit [30] — BUSY: Indicates that DecMul is still processing the operation. The data currently read must not be considered valid and the Master should read the word again later.

Bit [31] — DONE: Indicates that DecMul has completed its processing, including the corresponding results for the other multiplication factors.

Read Word 0101

Read Word 0101 : Get **x125** or **x1000** MSB (when required)

31	14	13	10	0
18 bits padded with zeros		11 Bits MSB Result of x125 or 14 Bits MSB Result of x1000		

BASE = 0 → Get ×125 MSB result

BASE = 1 → Get ×1000 MSB result

Bits [13:0] — MSB: Contains the remaining MSB portion of the ×125 or ×1000 product. The number of valid bits depends on the selected BASE: 11 bits for ×125 or 14 bits for ×1000.

Bits [31:10] — Zero padding: 22 bits fixed to 0 and carrying no information.

Output Words from DecMul (4/4)

Read Word 0110

Read Word 0110 : Retrieve Captured Operand



Returns the operand captured by the DecScale and **associated with the currently available set of results**.

Bits [31:0] — Captured Operand: Returns the 32-bit operand actually captured and processed by the DecScale.

Note : This word allows the Master to retrieve the captured operand if the original input value is no longer available to it. It also provides a direct means of verifying that the result set corresponds to the expected operand, since all DecScale results are generated from, and remain associated with, the captured operand.

Read Word 0111

Read Word 0111 : Get Two's Complement of Captured Operand



Bits [31:0] — Returns the 32-bit two's complement of captured operand

Output Words from DecDiv (1/4)

Read Word 1000

Read Word 1000 : Get /5 or /10 integer quotient

31	30	29	28	0																	
Done Flag		Busy Flag		30 Bits Integer Result of /5 or 29 Bits Integer Result of /10																	

BASE = 0 → Get /5 integer result

BASE = 1 → Get /10 integer result

Bits [29:0] — Integer quotient: Contains the 30-bit integer quotient of the /5 or /10 division, depending on the selected BASE.

Bit [30] — BUSY: Indicates that DecDiv is still processing the division. The response is not yet valid and the Master must read this word again later.

Bit [31] — DONE: Indicates that all divisions performed by DecDiv have completed and that their corresponding results are valid.

Read Word 1001

Read Word 1001 : Get /5 or /10 fractional quotient and extra precision for /125 or /1000

31	19	16	15	8	7	0
4Bits [19:16] pattern repeat indefinitely beyond 16-bit precision						
16 Bits fractional part of /5 or /10 results			8 bits of zeros		8 Bits additional fractional precision beyond 32 bits for /125 or /1000	

BASE = 0 → Get /5 fractional result
and 8-bit additional fractional precision for /125

BASE = 1 → Get /10 fractional result
and 8-bit additional fractional precision for /1000

Bits [7:0] — Additional fractional precision: Contains 8 additional fractional bits beyond the 32-bit fractional result provided by Read Word 1101, extending the precision of the /125 or /1000 quotient, depending on the selected BASE.

Bits [15:8] — Zero padding: 8 bits fixed to 0 and carrying no information.

Bits [31:16] — Fractional quotient: Contains the fractional part of the /5 or /10 quotient, depending on the selected BASE.

Within this 16-bit fractional field, bits [19:16] form a 4-bit repeating pattern. These four bits can therefore be repeated indefinitely to extend the fractional precision beyond the available 32-bit result.

Output Words from DecDiv (2/4)

Read Word 1010

Read Word 1010 : Get /25 or /100 integer quotient

31	30	29	28	27	25	0
Done Flag	Busy Flag	2 bits 00	28 Bits Integer Result of /25 or 26 Bits Integer Result of /100			

BASE = 0 → Get /25 integer result

BASE = 1 → Get /100 integer result

Bits [27:0] — Integer quotient: Contains the 30-bit integer quotient of the /25 or /100 division, depending on the selected BASE.

Bits [29:28] — Zero padding: 2 bits fixed to 0 and carrying no information

Bit [30] — BUSY: Indicates that DecDiv is still processing the division. The response is not yet valid and the Master must read this word again later.

Bit [31] — DONE: Indicates that all divisions performed by DecDiv have completed and that their corresponding results are valid.

Read Word 1011

Read Word 1011 : Get /25 or /100 fractional quotient

31	19	0
20Bits [19:0] pattern repeat indefinitely beyond 32-bit precision		
32 Bits fractional part of /25 or /100 results		

BASE = 0 → Get /25 fractional result

BASE = 1 → Get /100 fractional result

Bits [31:0] — Fractional quotient: Contains the 32-bit fractional part of the /25 or /100 quotient, depending on the selected BASE.

Within this 32-bit fractional result, bits [19:0] form a 20-bit repeating pattern. These 20 bits can therefore be repeated indefinitely to extend the fractional precision beyond the available 32-bit result.

Output Words from DecDiv (3/4)

Read Word 1100

Read Word 1100 : Get /125 or /1000 integer quotient

31	30	29	26	25	22	0
----	----	----	----	----	----	---

Done Flag	Busy Flag	4 bits 00..	26 Bits Integer Result of /125 or 23 Bits Integer Result of /1000
-----------	-----------	-------------	---

BASE = 0 → Get /25 integer result

BASE = 1 → Get /100 integer result

Bits [25:0] — Integer quotient: Contains the integer quotient of the /125 or /1000 division, depending on the selected BASE. The quotient occupies 26 bits for /25 and 23 bits for /100.

Bits [29:26] — Zero padding: 4 bits fixed to 0 and carrying no information.

Bit [30] — BUSY: Indicates that DecDiv is still processing the division. The response is not yet valid and the Master must read this word again later.

Bit [31] — DONE: Indicates that all divisions performed by DecDiv have completed and that their corresponding results are valid.

Read Word 1101

Read Word 1101 : Get /125 or /1000 fractional quotient

31	0
----	---

32 Bits fractional part of /125 or /1000 results (8 additional bits beyond 32 bits are provided in Word 1001)

BASE = 0 → Get /125 fractional result

BASE = 1 → Get /1000 fractional result

Bits [31:0] — Fractional quotient: Contains the 32-bit fractional part of the /125 or /1000 quotient, depending on the selected BASE.

Output Words from DecDiv (4/4)

Read Word 1110

Read Word 1110 : Get /3 integer quotient

31	30	0
----	----	---

Done Flag	31 Bits Integer Result of /3
-----------	------------------------------

Note: Because the quotient integer occupies 31 bits, this **word does not include a BUSY flag**, unlike the other integer-result words.

Bits [30:0] — Integer quotient: Contains the 31-bit integer quotient of the /3 division.

Bit [31] — DONE: Indicates that all divisions performed by DecDiv have completed and that their corresponding results are valid.

Remark: No BUSY flag is provided for this word because the integer quotient of /3 occupies 31 bits. The remaining single bit, bit [31], is therefore reserved for the DONE flag.

Read Word 1111

Read Word 1111 : Get /3 fractional quotient and division remainders

31	23	20	19	17	16	10	9	5	4	2	1	0
----	----	----	----	----	----	----	---	---	---	---	---	---

4Bits [23:20] pattern repeat indefinitely beyond 12-bit precision

16 Bits fractional part of /3	3 bits of zeros	7 Bits MOD /125	5 Bits MOD /25	3 Bits MOD /5	2 Bits MOD /3
-------------------------------	-----------------	-----------------	----------------	---------------	---------------

Remark: MOD /N denotes the remainder of division by N

Bits [1:0] — MOD /3: Contains the 2-bit remainder of the /3 division.

Bits [4:2] — MOD /5: Contains the 3-bit remainder of the /5 division.

Bits [9:5] — MOD /25: Contains the 5-bit remainder of the /25 division.

Bits [16:10] — MOD /125: Contains the 7-bit remainder of the /125 division.

Bits [19:17] — Zero padding: 3 bits fixed to 0 and carrying no information.

Bits [31:20] — Fractional quotient: Contains 12 fractional bits of the /3 quotient.

Within this fractional field, bits [23:20] form a 4-bit repeating pattern. These four bits can therefore be repeated indefinitely to extend the fractional precision beyond the available 12-bit result.

Operational Protocol

The DecScale operates according to a simple capture-and-consult protocol. The Master does not issue individual commands for each arithmetic operation. Instead, the DecScale captures one operand, generates its predefined result set, and retains the captured operand and associated TAG until the Master explicitly re-arms the circuit for a new calculation.

Two fundamental characteristics define the interaction between the Master and the DecScale.

1. Continuous Result Access

The Master can read the DecScale output at any time, regardless of the current state of the circuit. The Master may therefore modify the SELECT_OUTPUT field of i_cmd and read the corresponding output word while the DecScale is waiting for an operand, processing its calculations, or has already completed them. Reading an output does not modify the internal state of the DecScale.

When a result is not yet available, the corresponding status flags indicate its current state. In particular, BUSY indicates that processing is still in progress, while DONE indicates that the corresponding result is valid.

The Master therefore does not need to wait for the DecScale to become idle before consulting its outputs.

2. Combined Command Word

The i_cmd input is a 16-bit command word combining several independent control fields: TAG, CLR, NO_IRQ, BASE, and SELECT_OUTPUT.

These fields can be combined within the same command word. The Master can therefore configure several aspects of the DecScale simultaneously, such as selecting an output word, selecting the numerical base, controlling interrupt signaling, and providing a user-defined TAG.

The interpretation and combination of these fields are left to the Master software according to the requirements of the application.

DecScale Operating States

The DecScale can be understood as operating in three fundamental states.

Ready to Capture

In the Ready to Capture state, the DecScale is authorized to capture the operand present in the Data Register.

In this state:

- CLR = 0;
- BUSY = 0;
- DONE = 0;

no operand is currently locked;

the DecScale waits for a non-zero operand to be presented.

When a valid operand is captured, the DecScale also captures the TAG associated with that operand. The captured operand and TAG then become associated with the complete result set generated by the DecScale.

Locked State

Immediately after capturing an operand, the DecScale becomes locked to the captured operand and associated TAG.

From this point onward, changes to the external Data Register have no effect on the current calculation. Likewise, changes to the TAG field in the external command register do not replace the TAG already captured.

The DecScale remains locked until the Master explicitly re-arms it using CLR.

While locked, the DecScale either:

- continues processing its predefined calculations, in which case the corresponding BUSY flag is asserted; or
- has completed the required processing, in which case the corresponding DONE flag is asserted.

The locked state therefore prevents the Master from unintentionally starting another calculation while the current result set is being generated or consulted.

The lock does not prevent result access. The Master remains free to change SELECT_OUTPUT and read any available output word associated with the captured operand.

CLR State

Asserting CLR = 1 places the DecScale in its clear/re-arm state.

In this state, the current captured operand and its generated result set are discarded. The

DecScale is no longer authorized to capture the operand present in the Data Register. The DecScale remains insensitive to changes in the external operand while CLR = 1. This provides the Master with a safe preparation phase during which the next operand and command fields can be configured without triggering a new calculation. When CLR is subsequently returned to 0, the DecScale becomes ready to capture the next valid operand present at its input.

Initial State

Following initialization, the interface is assumed to contain:

i_operand = 0x00000000

i_cmd = 0x0000

The DecScale is then in the Ready to Capture state.

Because the initial operand is 0x00000000, the 16 output words return:
0x00000000

The Master may nevertheless select any output word at any time by modifying SELECT_OUTPUT; until a valid operand has been captured, the returned value remains zero.

If the Master does not want the DecScale to capture the next non-zero operand immediately, it can assert CLR = 1. The DecScale will then remain insensitive to changes in the Data Register until CLR is released.

Capturing a New Operand

In the Ready to Capture state, the DecScale waits for a non-zero operand.

Once such an operand is presented, the DecScale captures it and simultaneously captures the associated TAG.

The captured values are then isolated from subsequent changes to the external registers:

External Data Register —→ Captured Operand

External TAG —→ Captured TAG

↓
Locked Result Set

The Master may subsequently modify the external operand register or other command fields without affecting the calculation already in progress.

The captured operand is therefore the sole operand used to generate the current DecScale result set.

Processing and Result Availability

Following operand capture, the DecScale generates its predefined multiplication and division results.

During processing, the appropriate BUSY flags indicate that the corresponding result data must not yet be considered valid.

The Master may still read the output register during this period. Reading an output word does not interrupt or modify the ongoing processing.

Once the relevant processing has completed, the corresponding DONE flag is asserted and the associated result becomes valid.

The o_irq signal provides an optional global completion notification. It is asserted when both DecMul and DecDiv have completed their processing. The NO_IRQ control field can be used to suppress this interrupt.

The interrupt therefore serves as a notification mechanism only; it does not determine whether the Master can access the output interface.

Result Consultation

Once an operand has been captured, the complete predefined result set is associated with that operand.

The Master accesses this result set through the SELECT_OUTPUT field of i_cmd. Changing SELECT_OUTPUT only changes which output word is returned through o_result; it does not restart or modify the calculation.

The Master may therefore consult several different results from the same captured operand in succession, for example:

- multiplication results;
- division results;
- fractional results;
- /3 results;
- remainders;
- the captured operand;
- the two's complement of the captured operand;
- the associated TAG.

The Master is not required to request these calculations individually. They belong to the predefined result set generated by the DecScale.

Command Changes During the Locked State

While the DecScale is locked, the Master may continue to modify command fields used for result consultation or configuration.

In particular, the Master may change:

- SELECT_OUTPUT, to select another output word;
- BASE, to select the base-5 or base-10 result set;
- NO_IRQ, to control interrupt signaling.

Changing these fields does not alter the captured operand or captured TAG.

The TAG remains associated with the current result set once captured.

CLR is the exception. Asserting CLR explicitly terminates the current calculation cycle, discards the current result set, and re-arms the DecScale for a new operand.

Re-arming for a New Calculation

To begin a new calculation, the Master must explicitly place the DecScale into the CLR state. The required sequence is:

1. Assert CLR = 1.
2. The current captured operand and result set are discarded.
3. Prepare the new operand while CLR remains asserted.
4. Prepare the required command fields and TAG.
5. Deassert CLR by setting CLR = 0.
6. The DecScale captures the operand and associated TAG.
7. The new predefined result set is generated.
8. The Master remains free to consult the resulting output words.

The operand must be prepared before CLR is released. Otherwise, the DecScale may capture whatever non-zero value happens to be present in the Data Register at the moment it becomes ready to capture.

The same principle applies to the TAG: the desired TAG must be present when the operand is captured if it is to be associated with that calculation.

Because the 16-bit command word combines several control fields, the Master may prepare TAG, BASE, SELECT_OUTPUT, and other command options while CLR = 1, thereby allowing several parameters to be established before the DecScale is released.

Protocol Summary

The DecScale protocol can therefore be summarized as:

Ready to Capture → Capture Operand & TAG → Lock → Process → BUSY/DONE → Consult Results → CLR → Prepare → Release CLR → Capture Next Operand

The essential rule is that the operand and TAG are captured once and remain locked to the generated result set.

The Master is subsequently free to consult that result set at any time by changing SELECT_OUTPUT, without re-triggering the arithmetic processing.

CLR is the explicit mechanism used to abandon the current result set and authorize the capture of a new operand.